

Tool: Python & R Programming

Register No: 192424189

Name: Dhanusha D

Date: 15-07-2026

Subject: Data Handling & Visualization

Code: DSA0604

1. Scenario 1: Smart City Air Quality Monitoring

Python Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    "Location": ["A", "B", "C", "D", "E", "F", "G", "H"],
    "PM25": [35, 60, 90, 45, 75, 50, 95, 40],
    "NO2": [20, 35, 50, 25, 40, 30, 55, 22],
    "CO": [0.4, 0.8, 1.2, 0.5, 1.0, 0.7, 1.4, 0.6],
    "Temperature": [30, 32, 35, 29, 33, 31, 36, 28],
    "Humidity": [60, 55, 45, 70, 50, 65, 40, 75]
}

df = pd.DataFrame(data)

plt.figure(figsize=(6, 5))

plt.scatter(df["PM25"], df["NO2"],
            c=df["PM25"],
            cmap="Reds",
            s=120)

plt.colorbar(label="PM2.5")

plt.title("Sequential Color Scale")

plt.xlabel("PM2.5")

plt.ylabel("NO2")

plt.show()

plt.figure(figsize=(6, 5))
```

```

plt.scatter(df["PM25"], df["Temperature"],
            c=df["Temperature"],
            cmap="coolwarm",
            s=120)

plt.colorbar(label="Temperature")

plt.title("Diverging Color Scale")

plt.xlabel("PM2.5")

plt.ylabel("Temperature")

plt.show()

plt.figure(figsize=(6,5))

sns.scatterplot(data=df,
                x="PM25",
                y="Humidity",
                hue="Location",
                palette="Set2",
                s=120)

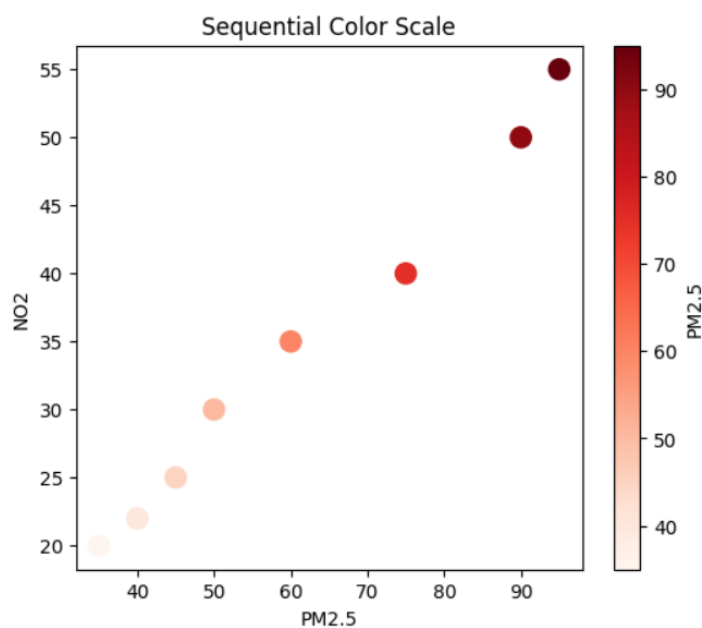
plt.title("Qualitative Color Scale")

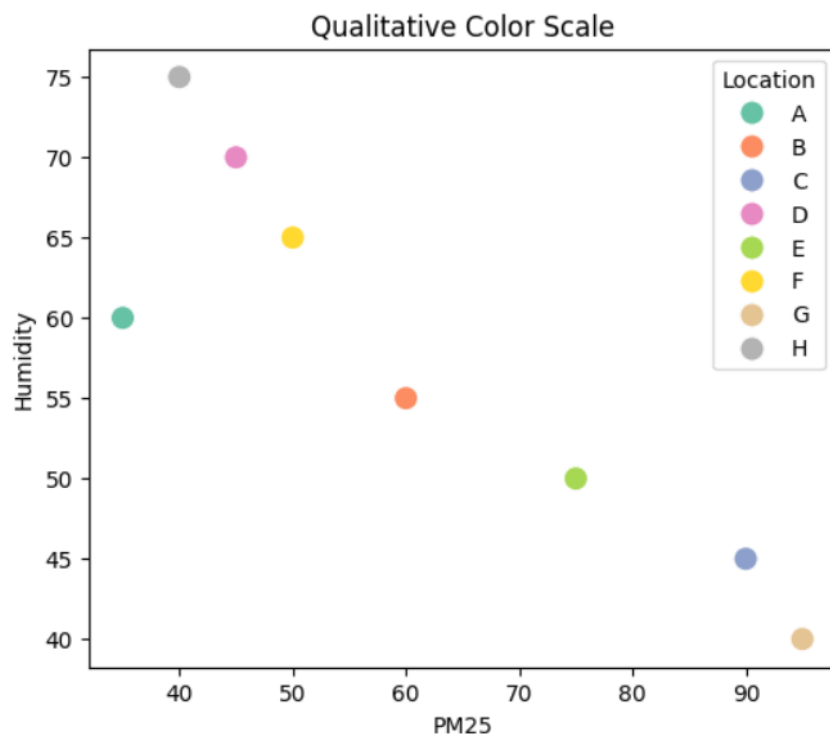
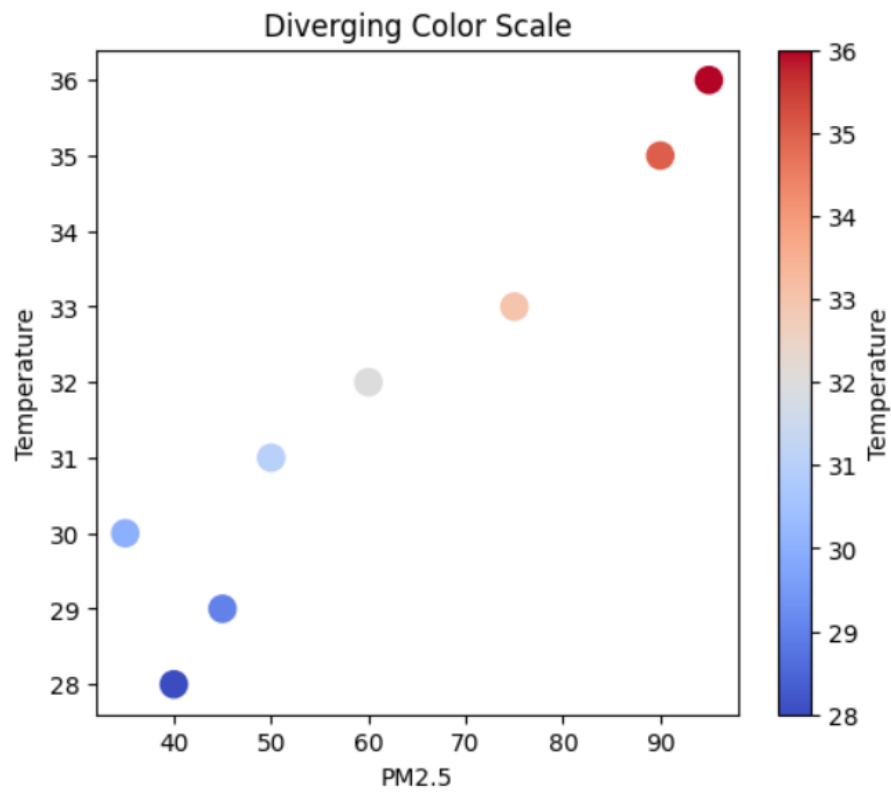
plt.show()

```

OUTPUT:

...





R PROGRAM:

```
library(ggplot2)
```

```
data <- data.frame(
```

```
Location=c("A","B","C","D","E","F","G","H"),
```

```

PM25=c(35,60,90,45,75,50,95,40),
NO2=c(20,35,50,25,40,30,55,22),
CO=c(0.4,0.8,1.2,0.5,1.0,0.7,1.4,0.6),
Temperature=c(30,32,35,29,33,31,36,28),
Humidity=c(60,55,45,70,50,65,40,75)
)

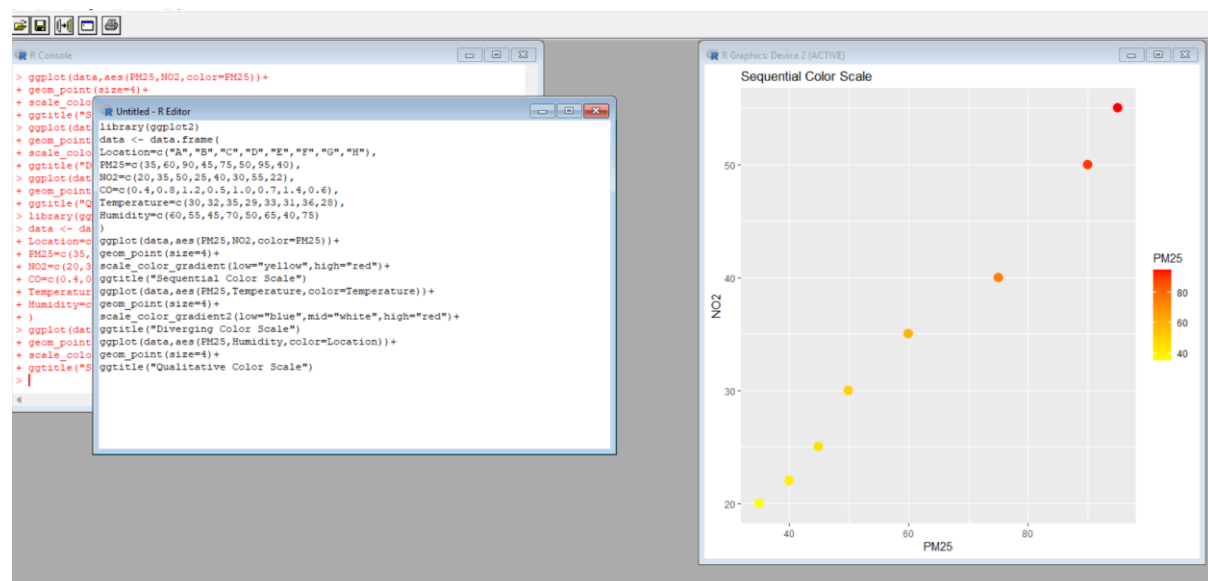
ggplot(data,aes(PM25,NO2,color=PM25))+
  geom_point(size=4)+
  scale_color_gradient(low="yellow",high="red")+
  ggtitle("Sequential Color Scale")

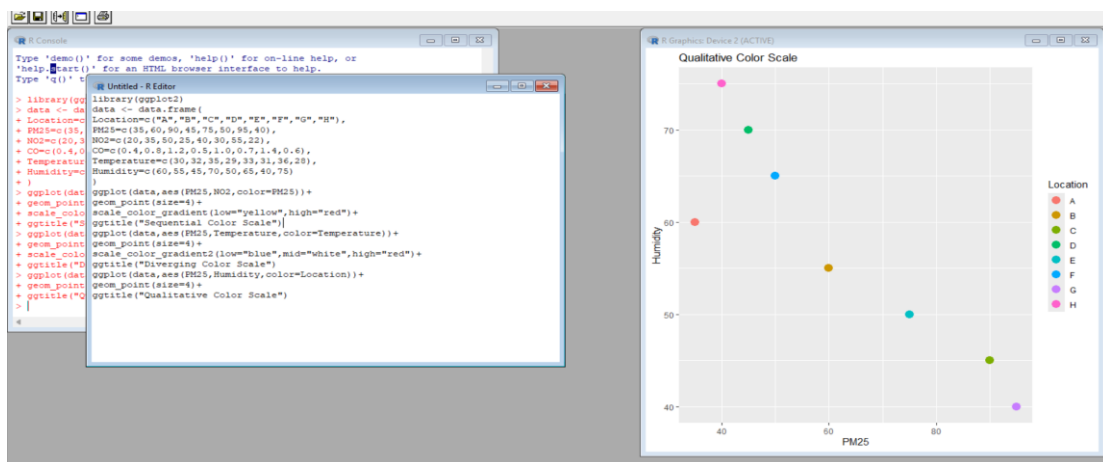
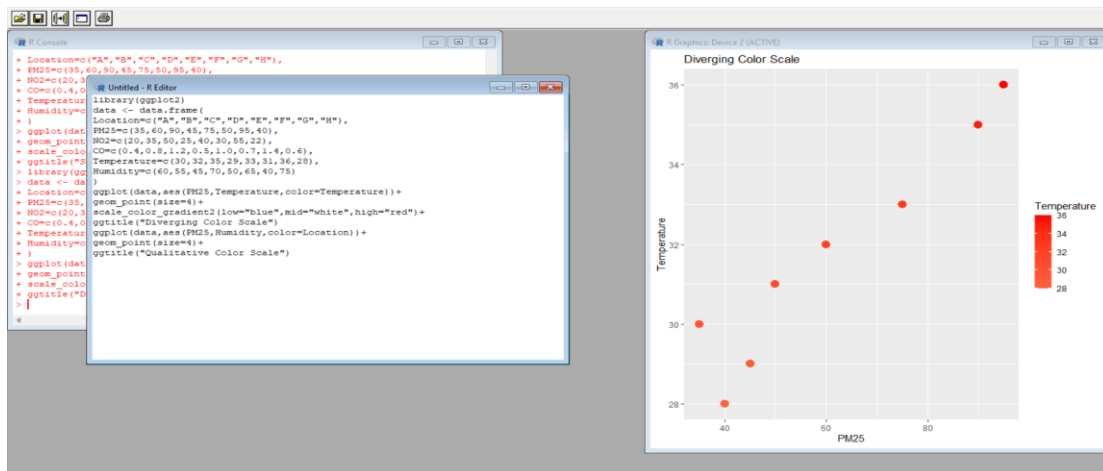
ggplot(data,aes(PM25,Temperature,color=Temperature))+
  geom_point(size=4)+
  scale_color_gradient2(low="blue",mid="white",high="red")+
  ggtitle("Diverging Color Scale")

ggplot(data,aes(PM25,Humidity,color=Location))+
  geom_point(size=4)+
  ggtitle("Qualitative Color Scale")

```

OUTPUT:





Scenario 2: Earthquake Risk Analysis

Python Program:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
data = {
    "Magnitude": [1.2, 2.5, 3.8, 4.6, 5.4, 6.2, 7.5, 8.4, 9.0],
    "Depth": [5, 12, 25, 50, 80, 150, 300, 500, 700],
    "Latitude": [10, 12, 14, 16, 18, 20, 22, 24, 26],
    "Longitude": [70, 72, 74, 76, 78, 80, 82, 84, 86],
    "Aftershock": [2, 4, 6, 8, 12, 18, 25, 35, 50]
}
```

```
df = pd.DataFrame(data)
```

```
plt.figure(figsize=(12,5))
```

```

plt.subplot(1,2,1)
plt.scatter(df["Magnitude"], df["Depth"],
            c=df["Aftershock"],
            cmap="viridis",
            s=100)

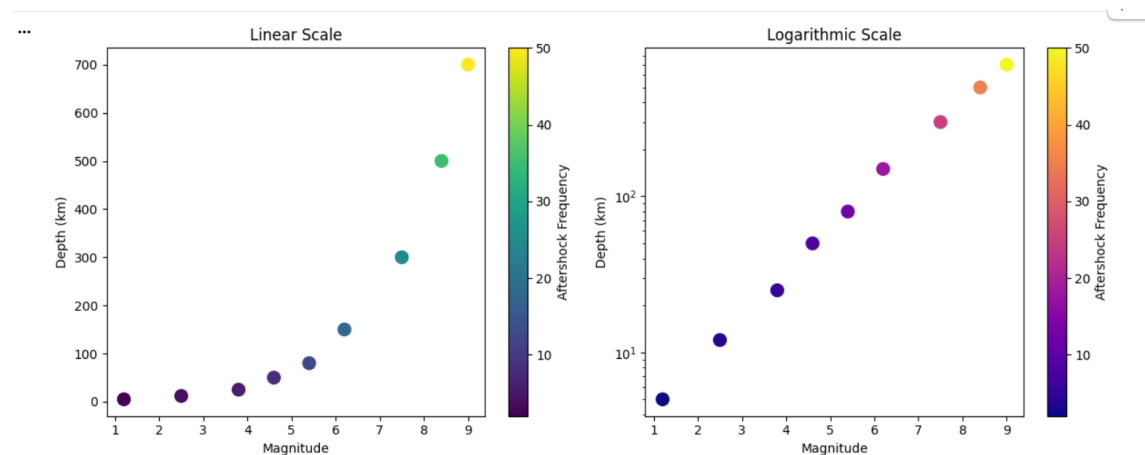
plt.colorbar(label="Aftershock Frequency")
plt.title("Linear Scale")
plt.xlabel("Magnitude")
plt.ylabel("Depth (km)")

plt.subplot(1,2,2)
plt.scatter(df["Magnitude"], df["Depth"],
            c=df["Aftershock"],
            cmap="plasma",
            s=100)

plt.yscale("log")
plt.colorbar(label="Aftershock Frequency")
plt.title("Logarithmic Scale")
plt.xlabel("Magnitude")
plt.ylabel("Depth (km)")
plt.tight_layout()
plt.show()

```

OUTPUT:



R PROGRAM:

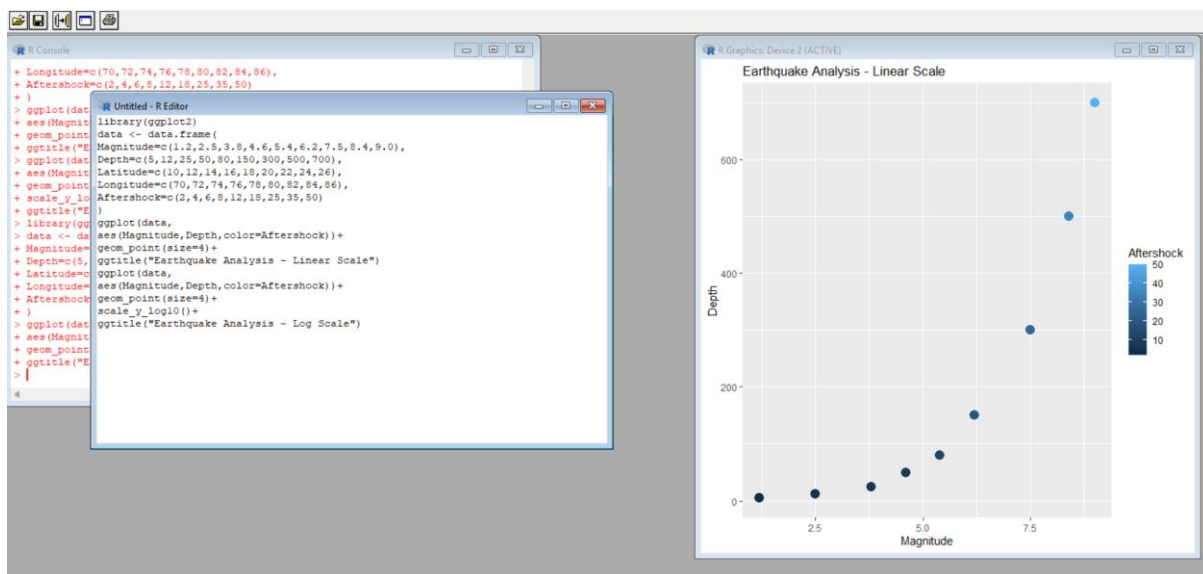
```
library(ggplot2)

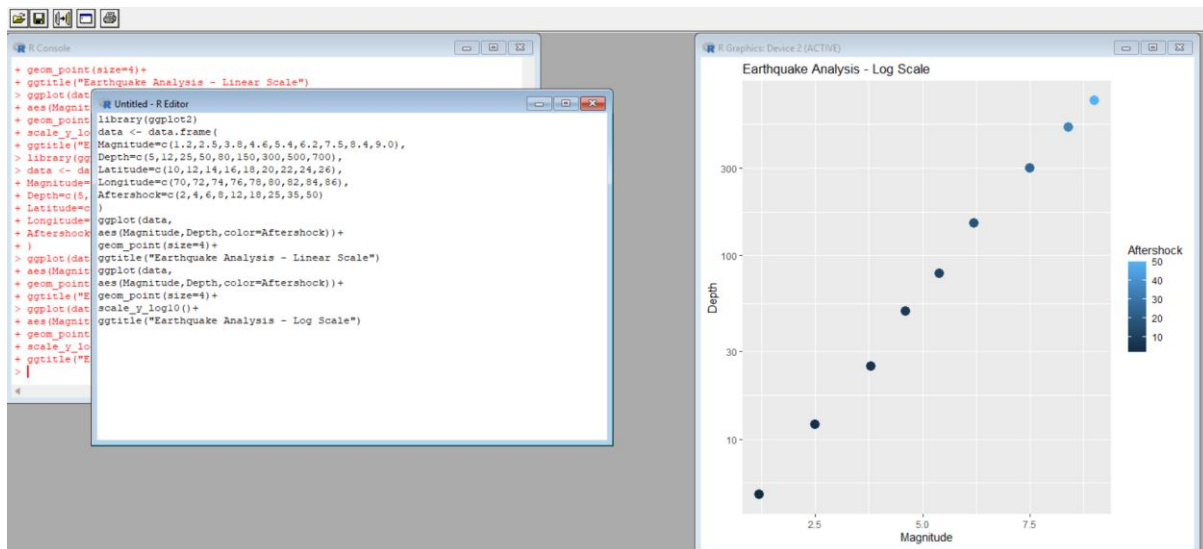
data <- data.frame(
  Magnitude=c(1.2,2.5,3.8,4.6,5.4,6.2,7.5,8.4,9.0),
  Depth=c(5,12,25,50,80,150,300,500,700),
  Latitude=c(10,12,14,16,18,20,22,24,26),
  Longitude=c(70,72,74,76,78,80,82,84,86),
  Aftershock=c(2,4,6,8,12,18,25,35,50)
)

ggplot(data,
  aes(Magnitude,Depth,color=Aftershock))+
  geom_point(size=4)+
  ggtitle("Earthquake Analysis - Linear Scale")

ggplot(data,
  aes(Magnitude,Depth,color=Aftershock))+
  geom_point(size=4)+
  scale_y_log10()+
  ggtitle("Earthquake Analysis - Log Scale")
```

OUTPUT:





Scenario 3: Wind Direction Analysis for Airport Operations

Python Program

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
wind_direction = [0, 30, 60, 90, 120, 150, 180, 210, 240, 270, 300, 330]
```

```
wind_speed = [12, 15, 18, 22, 20, 16, 14, 10, 13, 19, 21, 17]
```

```
theta = np.radians(wind_direction)
```

```
plt.figure(figsize=(12,5))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(wind_direction, wind_speed, marker='o')
```

```
plt.title("Cartesian Plot")
```

```
plt.xlabel("Wind Direction (Degrees)")
```

```
plt.ylabel("Wind Speed (km/h)")
```

```
plt.grid(True)
```

```
plt.subplot(1,2,2, projection='polar')
```

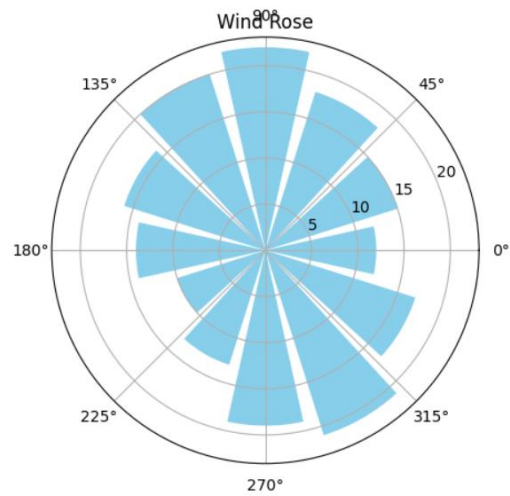
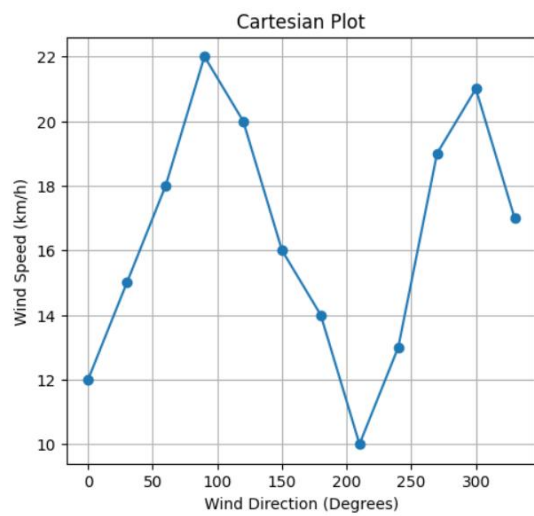
```
plt.bar(theta, wind_speed, width=np.radians(25), color='skyblue')
```

```
plt.title("Wind Rose")
```

```
plt.show()
```

OUTPUT:

...



R PROGRAM:

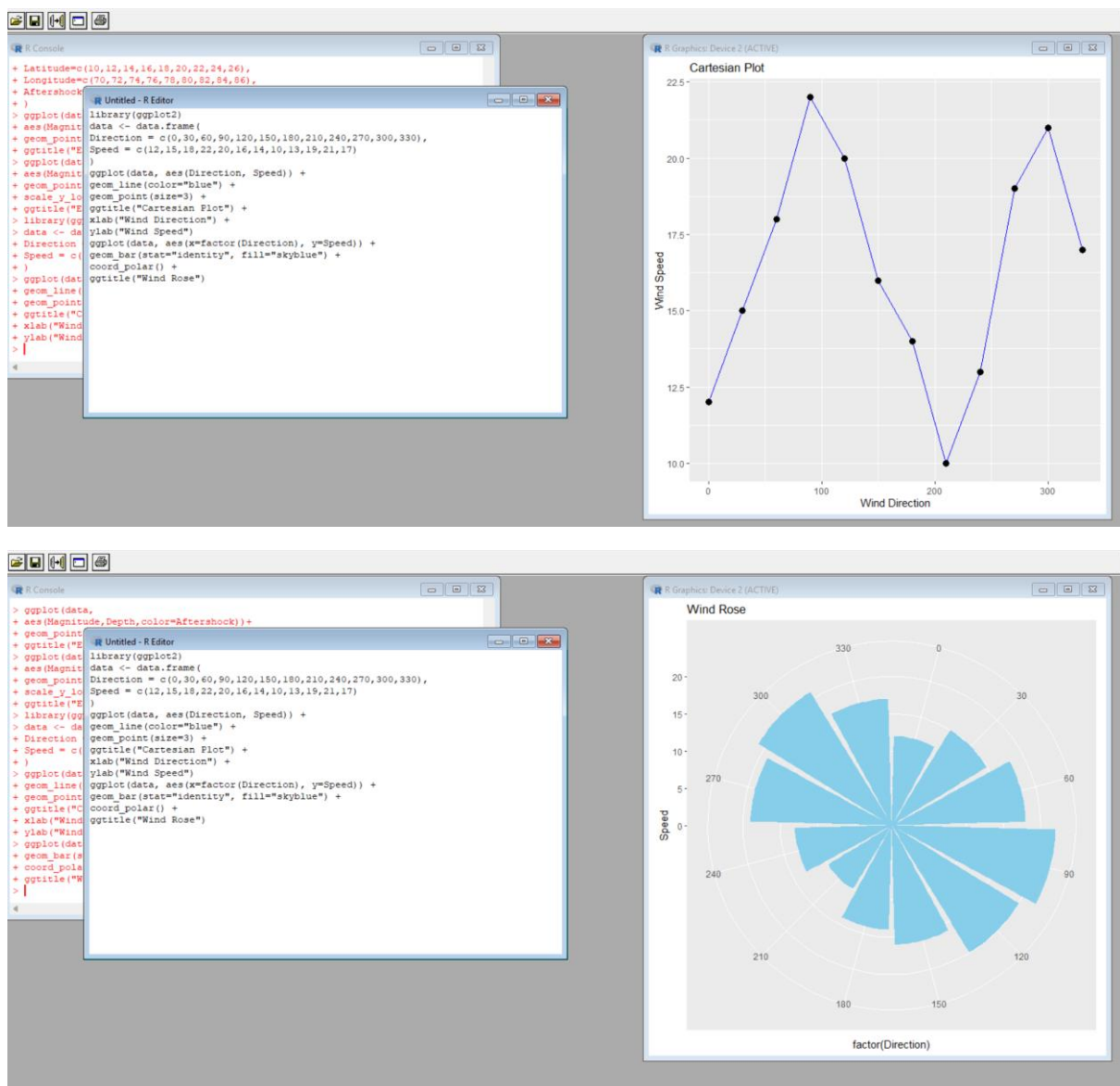
```
library(ggplot2)

data <- data.frame(
  Direction = c(0,30,60,90,120,150,180,210,240,270,300,330),
  Speed = c(12,15,18,22,20,16,14,10,13,19,21,17)
)

ggplot(data, aes(Direction, Speed)) +
  geom_line(color="blue") +
  geom_point(size=3) +
  ggtitle("Cartesian Plot") +
  xlab("Wind Direction") +
  ylab("Wind Speed")

ggplot(data, aes(x=factor(Direction), y=Speed)) +
  geom_bar(stat="identity", fill="skyblue") +
  coord_polar() +
  ggtitle("Wind Rose")
```

OUTPUT:



Scenario 4: Hospital Patient Monitoring Dashboard

Python Program

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
data = {
```

```
    "Patient": ["P1", "P2", "P3", "P4", "P5"],
```

```
    "HeartRate": [75, 120, 65, 105, 85],
```

```
    "Oxygen": [98, 90, 96, 88, 99],
```

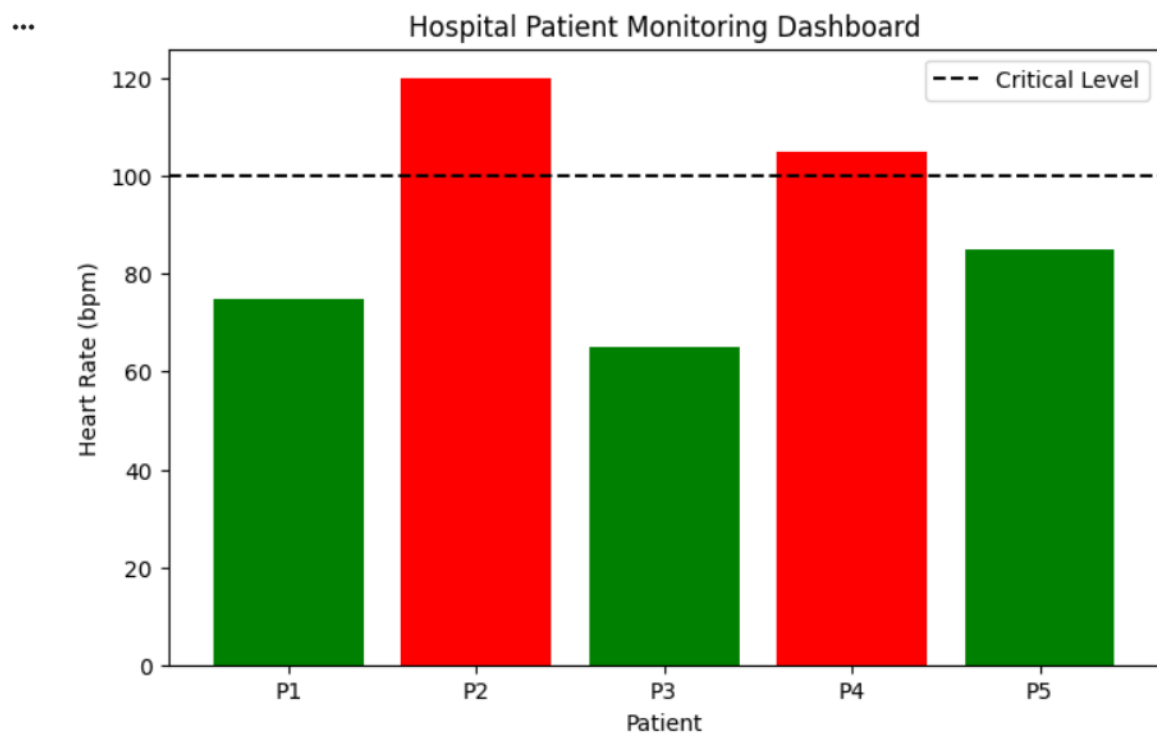
```
    "Temperature": [36.8, 38.9, 37.0, 39.5, 36.5]
```

```

}
df = pd.DataFrame(data)
colors = []
for hr in df["HeartRate"]:
    if hr > 100:
        colors.append("red")
    else:
        colors.append("green")
plt.figure(figsize=(8,5))
plt.bar(df["Patient"], df["HeartRate"], color=colors)
plt.axhline(100, color="black", linestyle="--", label="Critical Level")
plt.title("Hospital Patient Monitoring Dashboard")
plt.xlabel("Patient")
plt.ylabel("Heart Rate (bpm)")
plt.legend()
plt.show()

```

OUTPUT:



R PROGRAM:

```
library(ggplot2)

data <- data.frame(

Patient=c("P1","P2","P3","P4","P5"),

HeartRate=c(75,120,65,105,85),

Oxygen=c(98,90,96,88,99),

Temperature=c(36.8,38.9,37.0,39.5,36.5)

)

ggplot(data,

aes(Patient, HeartRate,

fill=HeartRate>100))+

geom_bar(stat="identity")+

scale_fill_manual(values=c("green","red"),

labels=c("Normal","Critical"),

name="Status")+

labs(

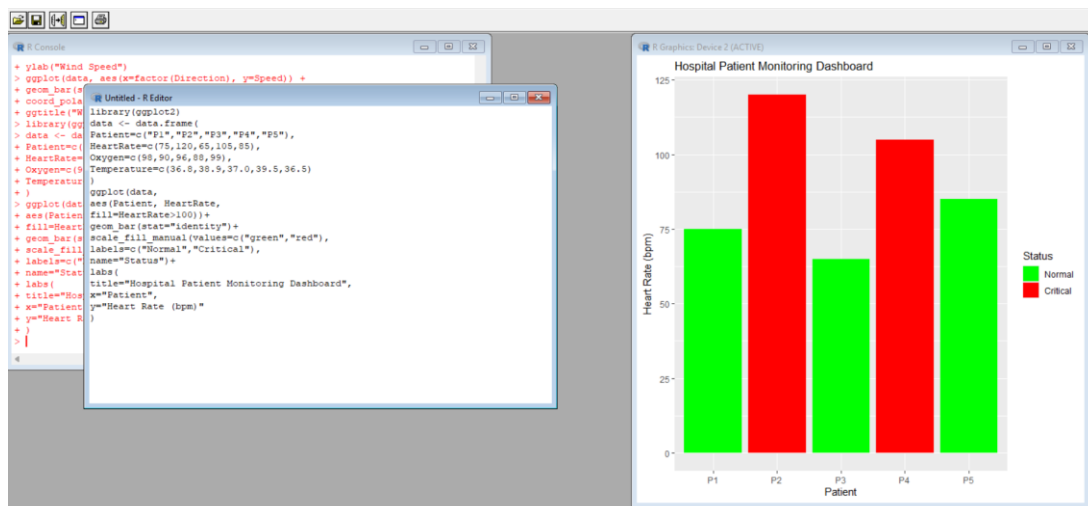
title="Hospital Patient Monitoring Dashboard",

x="Patient",

y="Heart Rate (bpm)"

)
```

OUTPUT:



Scenario 5: Global Climate Change Study

Python Program

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = pd.DataFrame({
    "India": [0.5, -0.3, 0.8],
    "USA": [-0.2, 0.4, 1.0],
    "China": [0.1, -0.5, 0.7],
    "Japan": [-0.4, 0.2, 0.6]
}, index=["2023", "2024", "2025"])

plt.figure(figsize=(5,4))

sns.heatmap(data, annot=True, cmap="coolwarm", center=0)

plt.title("Heatmap - Coolwarm")

plt.show()

plt.figure(figsize=(5,4))

sns.heatmap(data, annot=True, cmap="RdBu", center=0)

plt.title("Heatmap - RdBu")

plt.show()

plt.figure(figsize=(5,4))

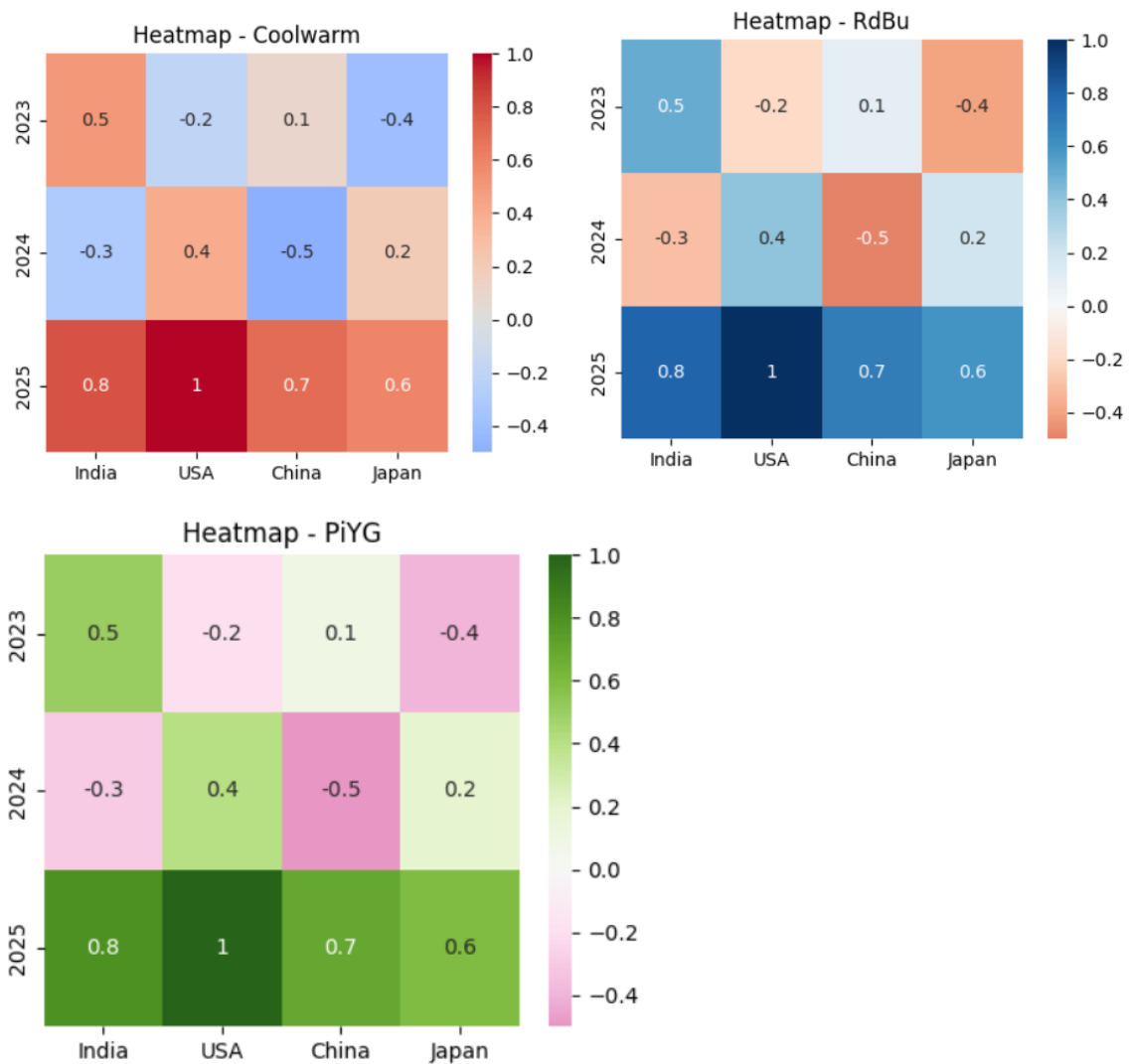
sns.heatmap(data, annot=True, cmap="PiYG", center=0)

plt.title("Heatmap - PiYG")

plt.show()
```

OUTPUT:

...



R PROGRAM:

```
library(ggplot2)
```

```
library(reshape2)
```

```
data <- data.frame(
```

```
Year=c("2023", "2024", "2025"),
```

```
India=c(0.5,-0.3,0.8),
```

```
USA=c(-0.2,0.4,1.0),
```

```
China=c(0.1,-0.5,0.7),
```

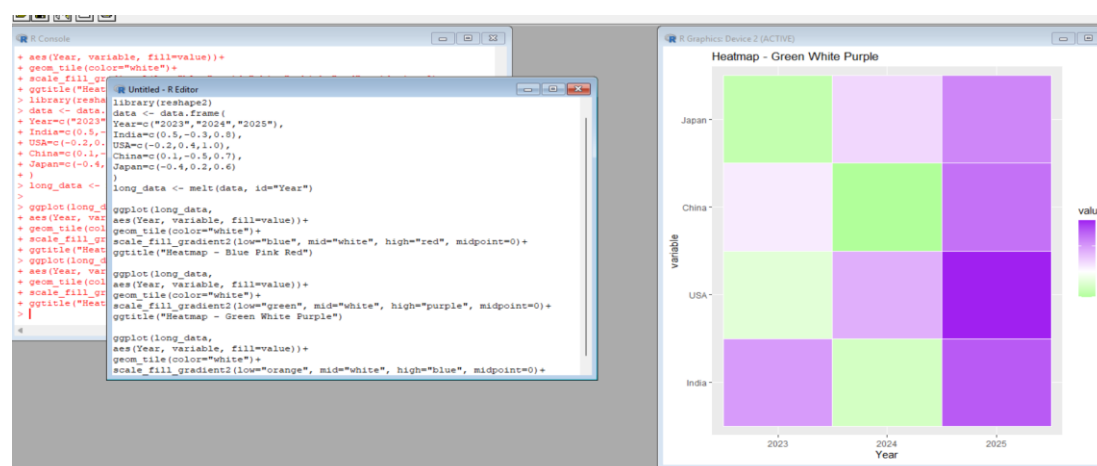
```
Japan=c(-0.4,0.2,0.6)
```

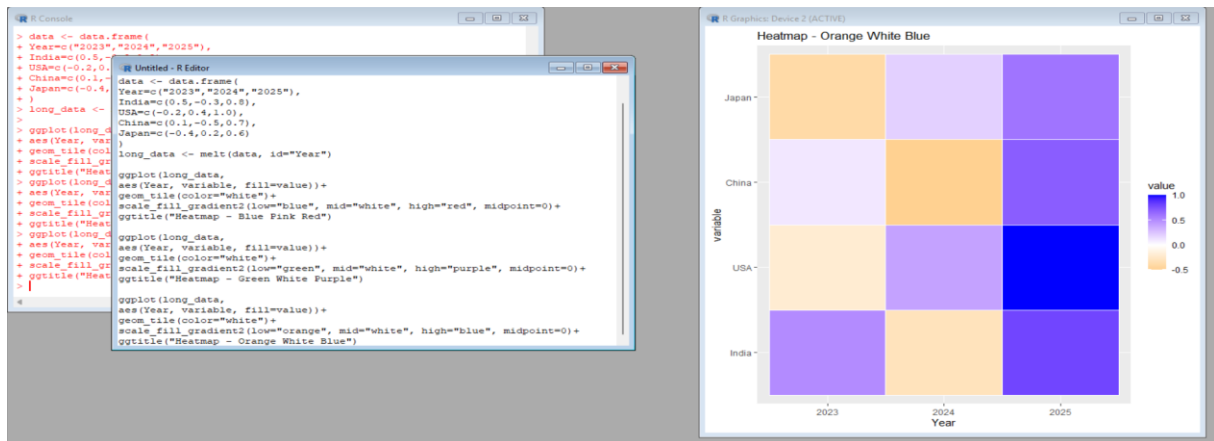
```
)
```

```
long_data <- melt(data, id="Year")
```

```
ggplot(long_data,
```

```
ggtitle("Heatmap - Orange White Blue")
```





Scenario 6: E-Commerce Sales Analytics

Python Program:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
data = {
```

"Category": ["Electronics", "Clothing", "Furniture", "Groceries", "Sports"],

"Region": ["North", "South", "East", "West", "Central"],

"Sales": [85000, 45000, 65000, 30000, 55000]

}

```
df = pd.DataFrame(data)
```

```
plt.figure(figsize=(8,5))
```

```
scatter = plt.scatter(df["Region"],
```

```
df["Sales"],
```

```
c=df["Sales"],
```

```
cmap="viridis",
```

s=250)

```
for i in range(len(df)):
```

```
plt.text(df["Region"][i],
```

```
df["Sales"][i],
```

```
df["Category"][i],
```

```
fontsize=9,
```

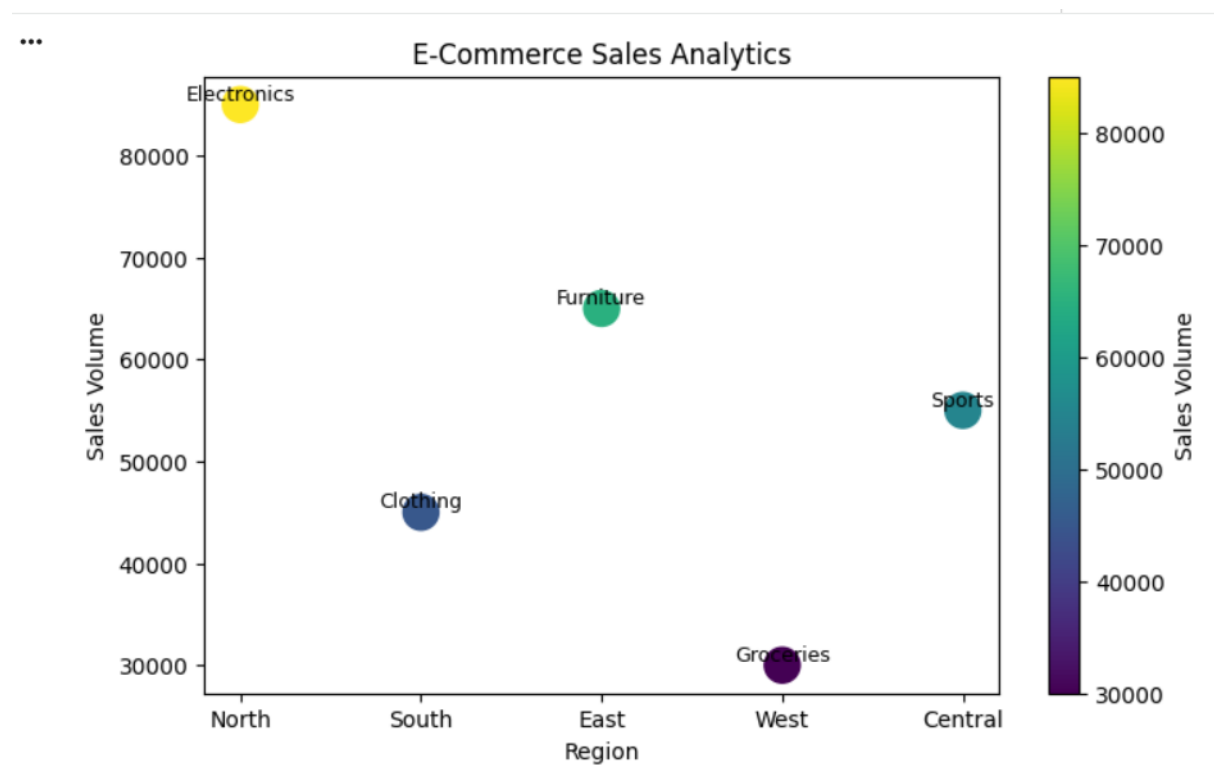
```
ha="center",
```

```

        va="bottom")
plt.colorbar(scatter,label="Sales Volume")
plt.title("E-Commerce Sales Analytics")
plt.xlabel("Region")
plt.ylabel("Sales Volume")
plt.show()

```

OUTPUT:



R PROGRAM:

```

library(ggplot2)
data <- data.frame(
  Category=c("Electronics","Clothing","Furniture","Groceries","Sports"),
  Region=c("North","South","East","West","Central"),
  Sales=c(85000,45000,65000,30000,55000)
)
ggplot(data,
  aes(x=Region,

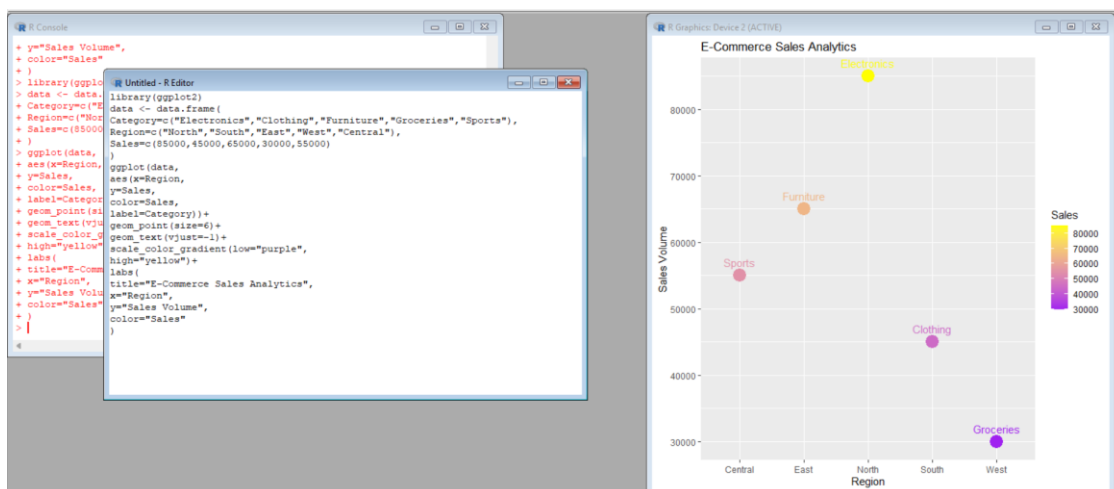
```

```

y=Sales,
color=Sales,
label=Category)))+
geom_point(size=6)+
geom_text(vjust=-1)+
scale_color_gradient(low="lightblue",
high="darkblue")+
labs(
title="E-Commerce Sales Analytics",
x="Region",
y="Sales Volume",
color="Sales"
)

```

OUTPUT:



Scenario 7: Satellite Image Vegetation Analysis

Python Program:

```

import pandas as pd

import matplotlib.pyplot as plt

data = {

    "Location": ["A", "B", "C", "D", "E", "F"],

    "NDVI": [-0.8, -0.2, 0.1, 0.4, 0.7, 0.9]

```

```

}

df = pd.DataFrame(data)

plt.figure(figsize=(6,4))

plt.scatter(df["Location"],
            df["NDVI"],
            c=df["NDVI"],
            cmap="Greens",
            s=250)

plt.colorbar(label="NDVI")

plt.title("Sequential Palette (Greens)")

plt.xlabel("Location")

plt.ylabel("NDVI")

plt.show()

plt.figure(figsize=(6,4))

plt.scatter(df["Location"],
            df["NDVI"],
            c=df["NDVI"],
            cmap="RdYlGn",
            s=250)

plt.colorbar(label="NDVI")

plt.title("Diverging Palette (Red-Yellow-Green)")

plt.xlabel("Location")

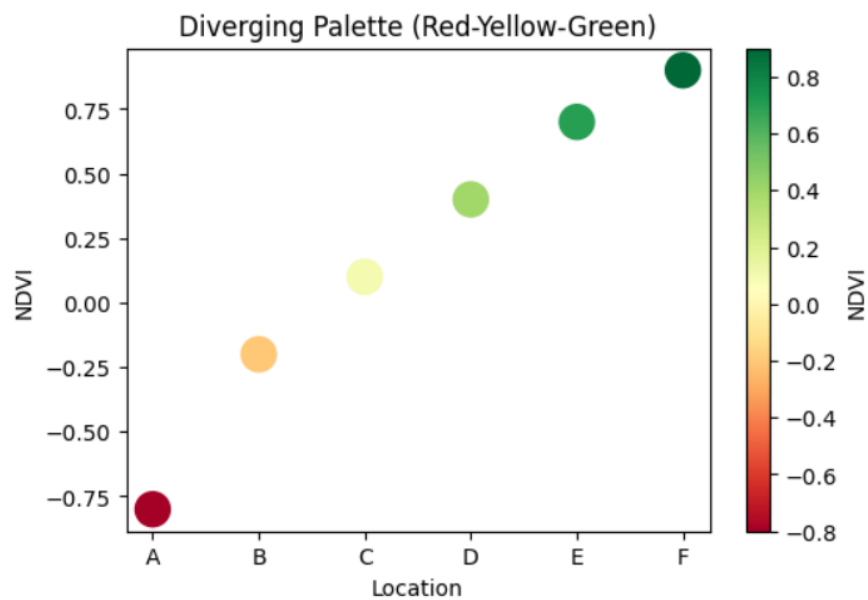
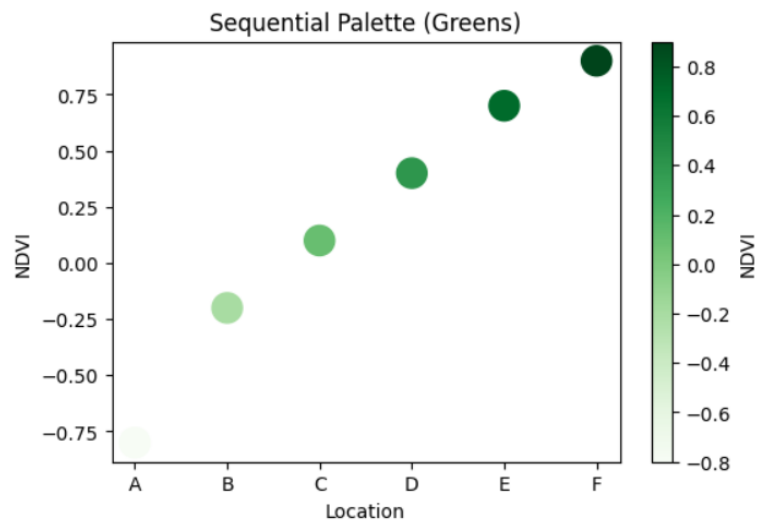
plt.ylabel("NDVI")

plt.show()

```

OUTPUT:

...



R PROGRAM:

```
library(ggplot2)

data <- data.frame(
  Location=c("A","B","C","D","E","F"),
  NDVI=c(-0.8,-0.2,0.1,0.4,0.7,0.9)
)

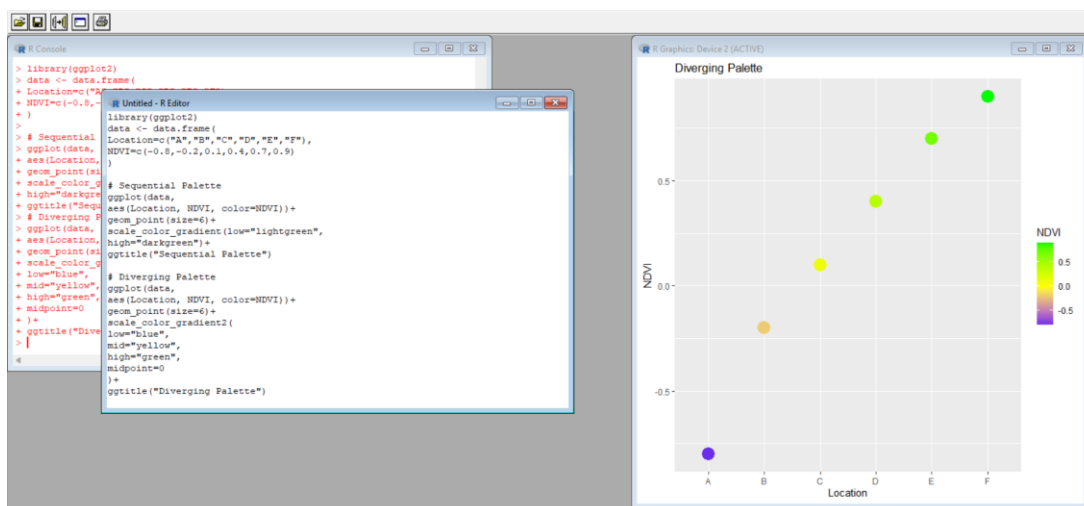
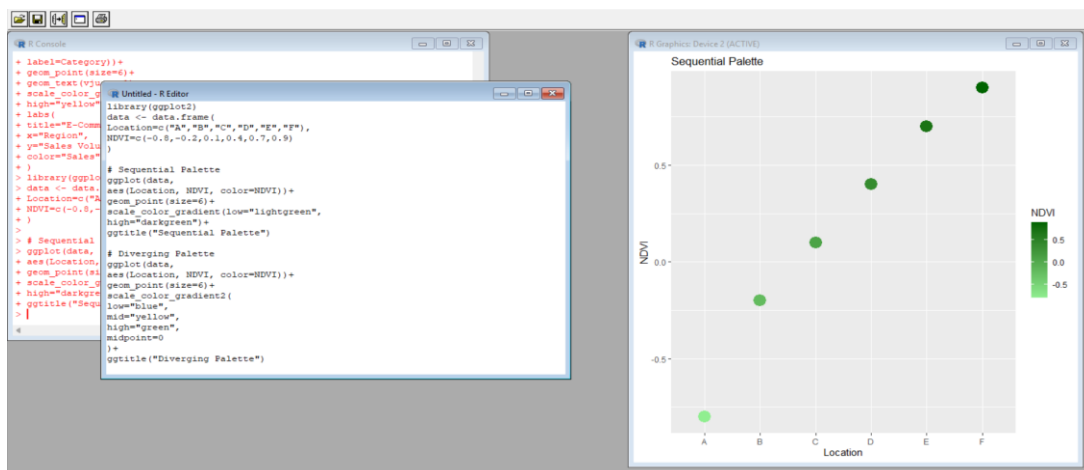
ggplot(data,
  aes(Location, NDVI, color=NDVI))+
  geom_point(size=6)+
  scale_color_gradient(low="lightgreen",
```

```

high="darkgreen")+
ggtitle("Sequential Palette")
ggplot(data,
aes(Location, NDVI, color=NDVI))+
geom_point(size=6)+
scale_color_gradient2(
low="blue",
mid="yellow",
high="green",
midpoint=0
)+
ggtitle("Diverging Palette")

```

OUTPUT:



Scenario 8: Cryptocurrency Market Volatility

Python Program:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {
    "Day": [1, 2, 3, 4, 5],
    "Bitcoin": [60000, 62000, 61000, 64000, 65000],
    "Ethereum": [3000, 3200, 3100, 3300, 3400],
    "Solana": [100, 120, 110, 130, 140],
    "Cardano": [0.5, 0.6, 0.55, 0.65, 0.70]
}

df = pd.DataFrame(data)

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)

plt.plot(df["Day"], df["Bitcoin"], label="Bitcoin", color="red")
plt.plot(df["Day"], df["Ethereum"], label="Ethereum", color="blue")
plt.plot(df["Day"], df["Solana"], label="Solana", color="green")
plt.plot(df["Day"], df["Cardano"], label="Cardano", color="orange")

plt.title("Linear Scale")

plt.xlabel("Day")

plt.ylabel("Price (USD)")

plt.legend()

plt.subplot(1,2,2)

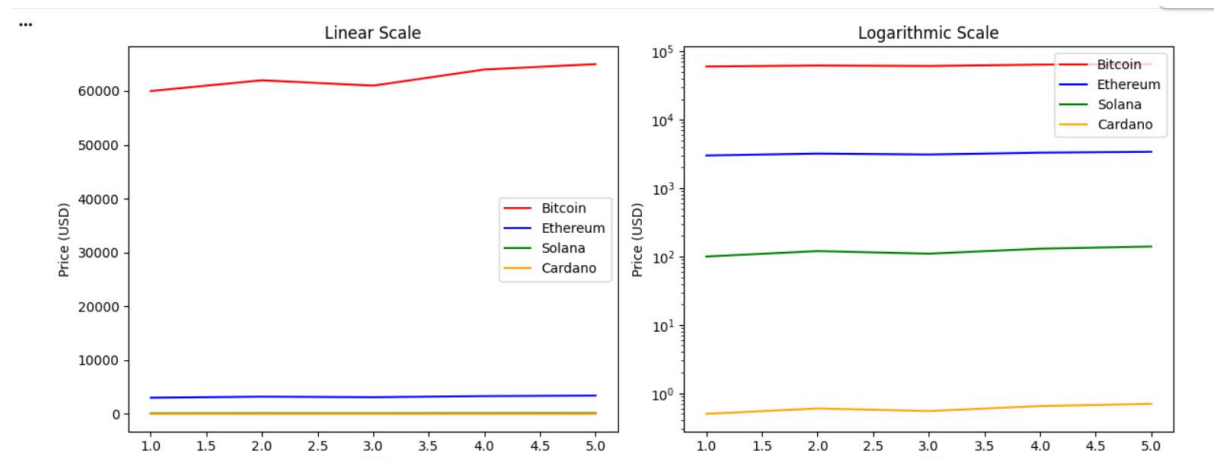
plt.plot(df["Day"], df["Bitcoin"], label="Bitcoin", color="red")
plt.plot(df["Day"], df["Ethereum"], label="Ethereum", color="blue")
plt.plot(df["Day"], df["Solana"], label="Solana", color="green")
plt.plot(df["Day"], df["Cardano"], label="Cardano", color="orange")

plt.yscale("log")

plt.title("Logarithmic Scale")
```

```
plt.xlabel("Day")
plt.ylabel("Price (USD)")
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT:



R PROGRAM:

```
library(ggplot2)

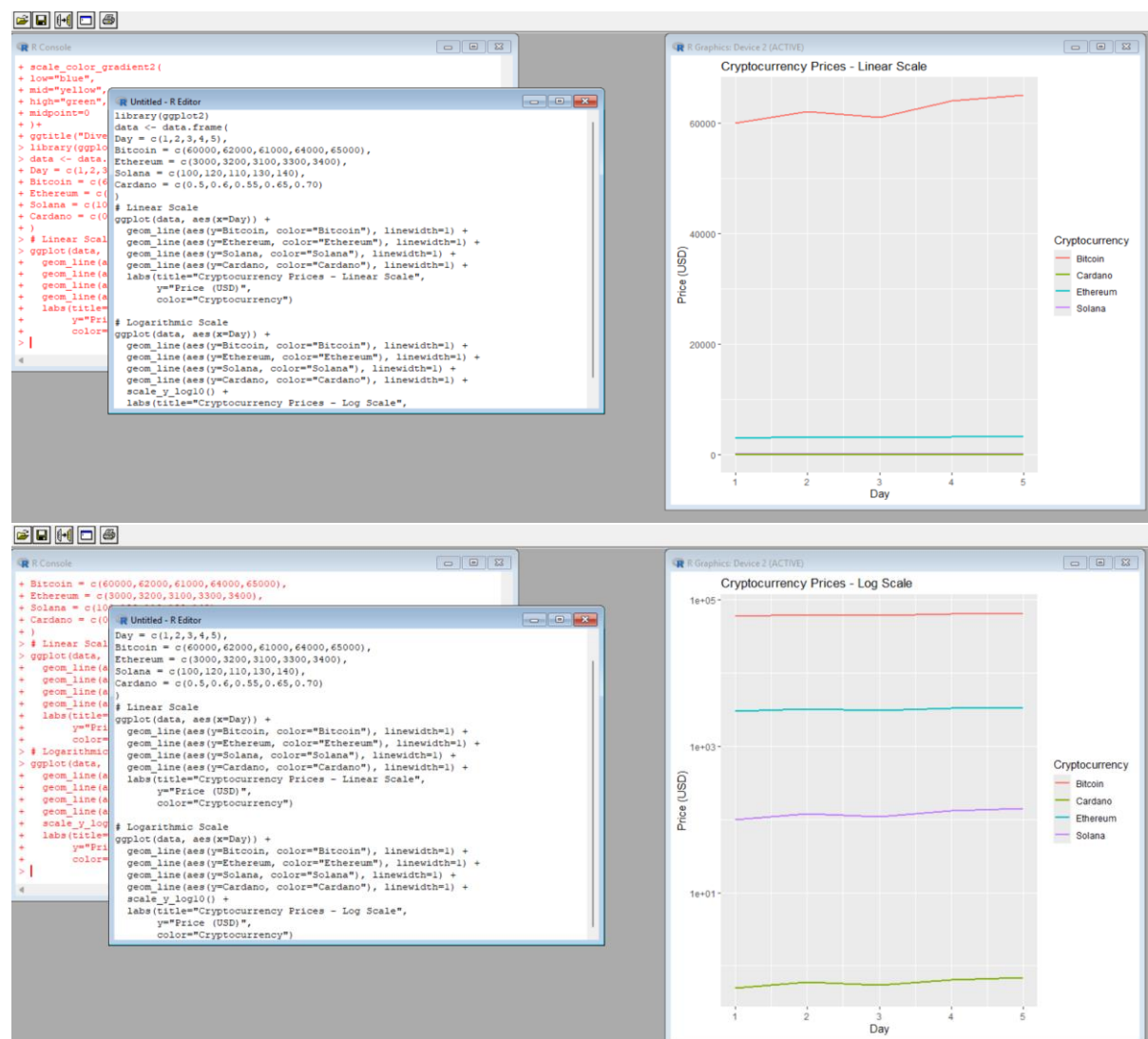
data <- data.frame(
  Day = c(1,2,3,4,5),
  Bitcoin = c(60000,62000,61000,64000,65000),
  Ethereum = c(3000,3200,3100,3300,3400),
  Solana = c(100,120,110,130,140),
  Cardano = c(0.5,0.6,0.55,0.65,0.70)
)

ggplot(data, aes(x=Day)) +
  geom_line(aes(y=Bitcoin, color="Bitcoin"), linewidth=1) +
  geom_line(aes(y=Ethereum, color="Ethereum"), linewidth=1) +
  geom_line(aes(y=Solana, color="Solana"), linewidth=1) +
  geom_line(aes(y=Cardano, color="Cardano"), linewidth=1) +
  labs(title="Cryptocurrency Prices - Linear Scale",
```

```
y="Price (USD)",
color="Cryptocurrency")
```

```
ggplot(data, aes(x=Day)) +
  geom_line(aes(y=Bitcoin, color="Bitcoin"), linewidth=1) +
  geom_line(aes(y=Ethereum, color="Ethereum"), linewidth=1) +
  geom_line(aes(y=Solana, color="Solana"), linewidth=1) +
  geom_line(aes(y=Cardano, color="Cardano"), linewidth=1) +
  scale_y_log10() +
  labs(title="Cryptocurrency Prices - Log Scale",
       y="Price (USD)",
       color="Cryptocurrency")
```

OUTPUT:



Scenario 9: Marathon Runner Performance Analysis

Python Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

data = {
    "Distance": [1, 2, 3, 4, 5, 6],
    "Speed": [10, 12, 15, 13, 11, 14],
    "Direction": [0, 60, 120, 180, 240, 300]
}

df = pd.DataFrame(data)
theta = np.radians(df["Direction"])

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)

scatter = plt.scatter(df["Distance"],
                      df["Speed"],
                      c=df["Speed"],
                      cmap="viridis",
                      s=120)

plt.colorbar(scatter, label="Speed")
plt.title("Cartesian Plot")
plt.xlabel("Distance (km)")
plt.ylabel("Speed (km/h)")

plt.subplot(1,2,2, projection="polar")
scatter = plt.scatter(theta,
                      df["Distance"],
                      c=df["Speed"],
                      cmap="viridis",
                      s=120)
```

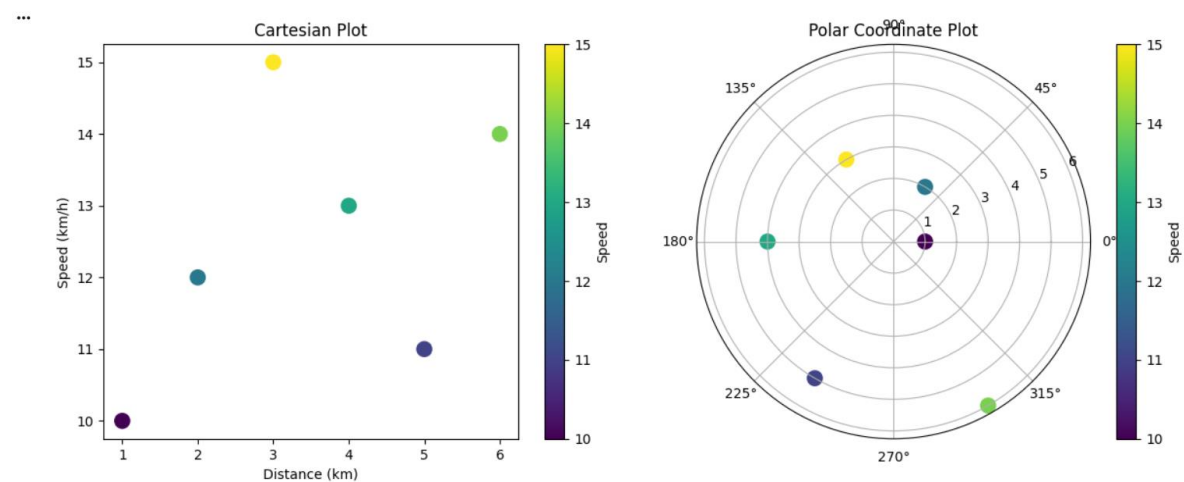
```
plt.colorbar(scatter, label="Speed")
```

```
plt.title("Polar Coordinate Plot")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



R PROGRAM:

```
library(ggplot2)
```

```
data <- data.frame(
```

```
Distance = c(1,2,3,4,5,6),
```

```
Speed = c(10,12,15,13,11,14),
```

```
Direction = c(0,60,120,180,240,300)
```

```
)
```

```
ggplot(data,
```

```
aes(x=Distance,
```

```
y=Speed,
```

```
color=Speed)) +
```

```
geom_point(size=4) +
```

```
geom_line() +
```

```
scale_color_gradient(low="blue", high="red") +
```

```
labs(title="Cartesian Plot",
```

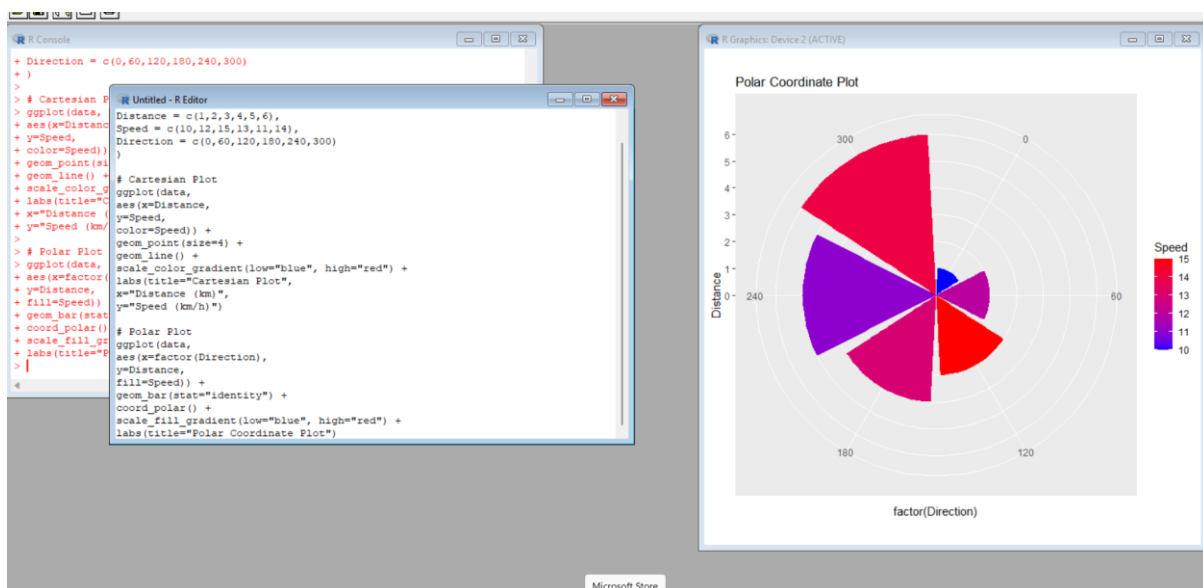
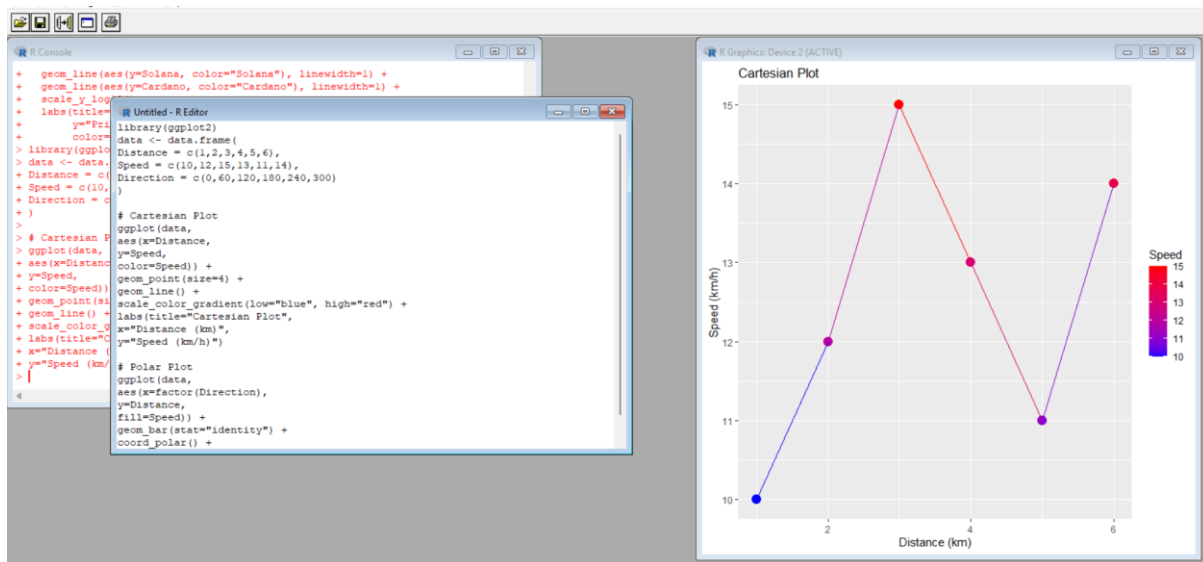
```
x="Distance (km)",
```

```

y="Speed (km/h)")
ggplot(data,
aes(x=factor(Direction),
y=Distance,
fill=Speed)) +
geom_bar(stat="identity") +
coord_polar() +
scale_fill_gradient(low="blue", high="red") +
labs(title="Polar Coordinate Plot")

```

OUTPUT:



Scenario 10: COVID-19 Vaccination Campaign Dashboard

Python Program:

```
import pandas as pd

import matplotlib.pyplot as plt

data = {
    "District": ["A", "B", "C", "D", "E"],
    "Vaccination": [65, 80, 55, 90, 70]
}

df = pd.DataFrame(data)

plt.figure(figsize=(6,4))

plt.bar(df["District"], df["Vaccination"],
        color="skyblue")

plt.title("Vaccination Dashboard - Blue Palette")

plt.xlabel("District")

plt.ylabel("Vaccination Rate (%)")

plt.show()

colors=[]

for rate in df["Vaccination"]:
    if rate >= 80:
        colors.append("green")
    elif rate >= 60:
        colors.append("yellow")
    else:
        colors.append("red")

plt.figure(figsize=(6,4))

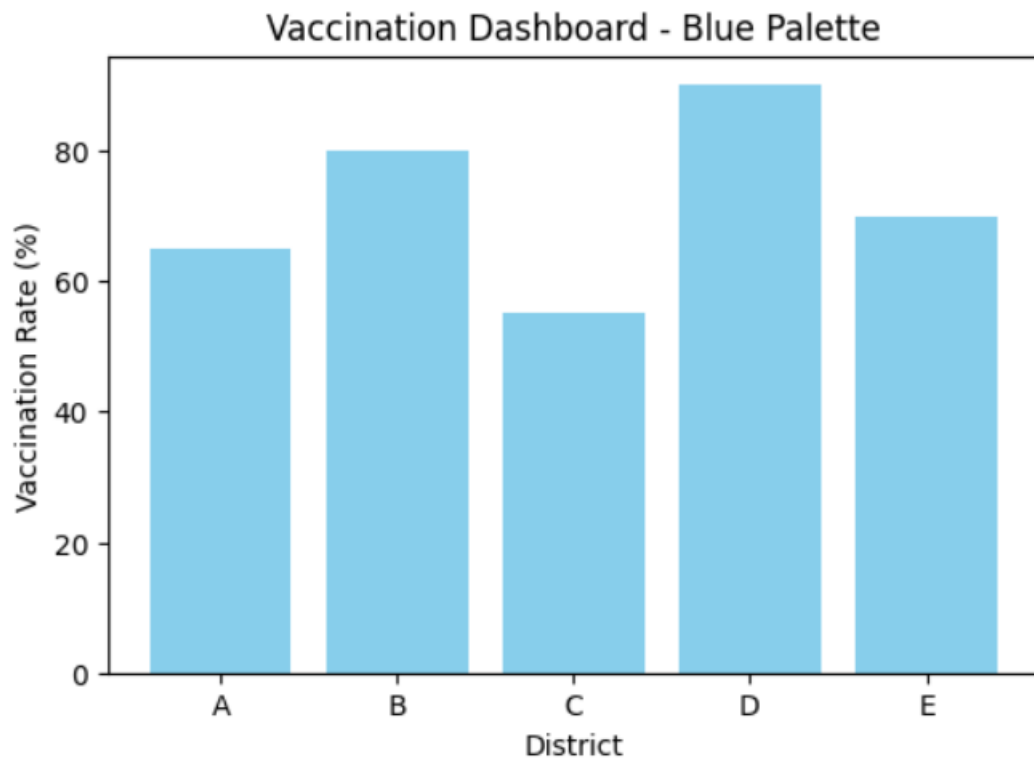
plt.bar(df["District"], df["Vaccination"],
        color=colors)

plt.title("Vaccination Dashboard - Highlighted")
```

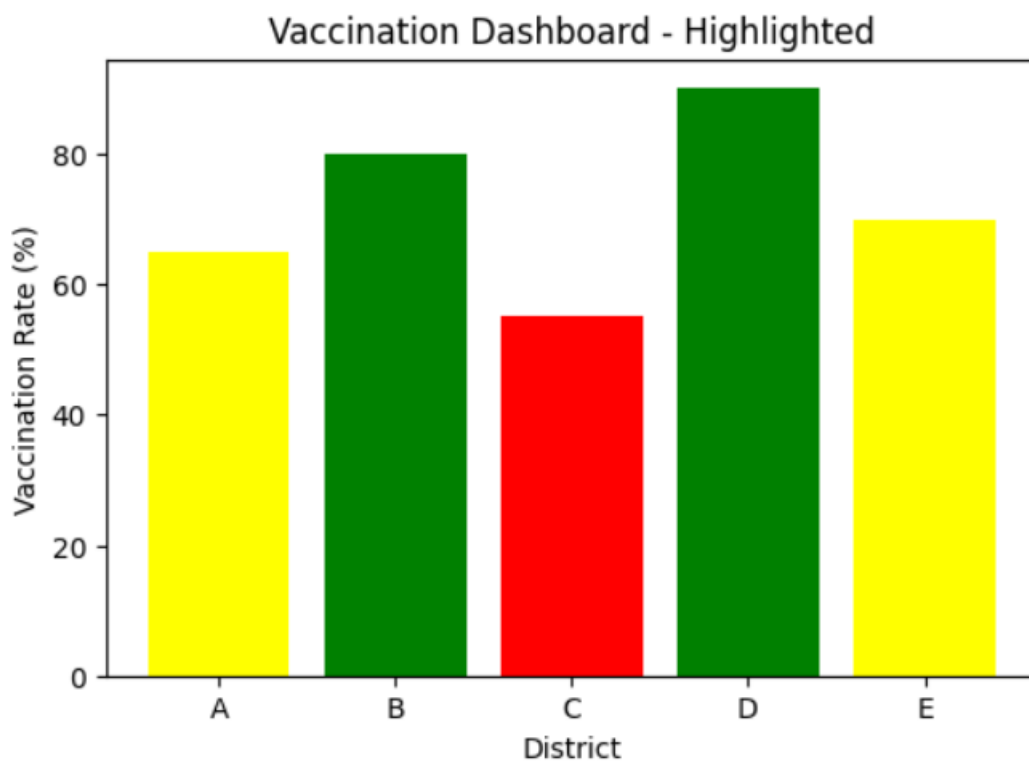
```
plt.xlabel("District")
plt.ylabel("Vaccination Rate (%)")
plt.show()
```

OUTPUT:

...



...



R PROGRAM:

```
library(ggplot2)

data <- data.frame(
  District=c("A","B","C","D","E"),
  Vaccination=c(65,80,55,90,70)
)

ggplot(data,
  aes(District,
  Vaccination,
  fill=Vaccination))+
  geom_bar(stat="identity")+
  scale_fill_gradient(low="lightblue",
  high="darkblue")+

  labs(
  title="Vaccination Dashboard - Blue Palette",
  y="Vaccination Rate (%)")
)

data$Status <- ifelse(data$Vaccination>=80,"High",
  ifelse(data$Vaccination>=60,"Medium","Low"))

ggplot(data,
  aes(District,
  Vaccination,
  fill=Status))+
  geom_bar(stat="identity")+
  scale_fill_manual(values=c(
  "High"="green",
  "Medium"="yellow",
  "Low"="red"))+
```

```
labs(
  title="Vaccination Dashboard - Highlighted",
  y="Vaccination Rate (%)"
)
```

OUTPUT:

